Semi-supervised Deep Transfer for Regression without Domain Alignment

Supplementary Material

A. Derivations

A.1. Mixtures of Gaussians and Exponentials

Here, we derive the algorithm to fit a mixture of exponential and Gaussian distributions. Given samples $\{\mathbf{x}_i\}_{i=1}^N$ from an unknown distribution p_x, s.t., $\mathbf{x}_i \in \mathbb{R}_+^d$. If negative components exist, we can add a constant offset to make the data non-negative. We model the underlying distribution by k_1 Gaussians and k_2 exponentials. The density of a mixture model p_θ, can be written as[1];

$$p_\theta(\mathbf{x}) = \sum_{i=1}^{k_1+k_2} p_\theta(z) p_\theta(\mathbf{x}|z) \tag{1}$$

, where z is the latent variable, s.t., $p_\theta(z) = Multi(z; \beta_1, \beta_2, ..., \beta_{k_1+k_2})$. The conditionals are defined as:

$$p_\theta(\mathbf{x}|z=i) = \begin{cases} \mathcal{N}(\mathbf{x}; \mu_i, \mathbf{\Sigma}_i) & i \in \{1, 2, ..., k_1\} \\ \text{Exp}(\mathbf{x}; \lambda_i) & i \in \{k_1+1, ..., k_1+k_2\} \end{cases}$$

We want to estimate the parameters $\theta = \{\{\beta_i\}_{i=1}^{k_1+k_2}, \{\mu_i, \mathbf{\Sigma}_i\}_{i=1}^{k_1}, \{\lambda_i\}_{i=k_1+1}^{k_2}\}$. To learn the parameters for mixture models, we maximize the Evidence Lower Bound (ELBO), using the expectation-maximization algorithm.

A.1.1. Expectation Step

Consider a variational distribution $q(z=j|\mathbf{x}_i) = q_{ji}$, i.e., the pmf that assigns probabilities to each datapoint to be in each of the chosen distributions. The optimization objective (ELBO – $F(\theta, q)$) can be written as:

$$F(\theta, q) = \mathop{\mathbb{E}}_{\mathbf{x} \in p_x} [\log p_\theta(\mathbf{x}) - \mathcal{D}_{KL}[q(z|\mathbf{x}) \,||\, p_\theta(z|\mathbf{x})]]$$

In the expectation step at the t-th iteration, we fix θ^t, and maximize $F(\theta^t, q)$, by setting the KL-divergence to be 0; i.e. $q(z|\mathbf{x}) = p_\theta(z|\mathbf{x})$. Thus,

$$q_{ji}^{t+1} = \begin{cases} \frac{\beta_j^t \mathcal{N}(\mathbf{x}_i; \mu_j^t, \mathbf{\Sigma}_j^t)}{M_{\theta^t}(\mathbf{x}_i)} & j \in \{1, 2, ..., k_1\} \\ \\ \frac{\beta_j^t \text{Exp}(\mathbf{x}_i; \lambda_j^t)}{M_{\theta^t}(\mathbf{x}_i)} & j \in \{k_1+1, ..., k_1+k_2\} \end{cases} \tag{2}$$

where,

$$M_{\theta^t}(\mathbf{x}_i) = \sum_{j=1}^{k_1} \beta_j^t \, \mathcal{N}(\mathbf{x}_i; \mu_j^t, \mathbf{\Sigma}_j^t) + \sum_{j=k_1+1}^{k_1+k_2} \beta_j^t \, \text{Exp}(\mathbf{x}_i; \lambda_j^t)$$

A.1.2. Maximization Step

Next, keeping q^{t+1} fixed, we maximize $F(\theta, q^t)$, such that the constraint $\sum_{i=1}^{k_1+k_2} \beta_i = 1$ is satisfied. Hence, solving the Lagrangian, we get the analytical solution to θ^{t+1}.

$$\beta_j^{t+1} = \frac{1}{N} \sum_{i=1}^{N} q_{ji}^{t+1}$$

$$\mu_j^{t+1} = \frac{1}{N \beta_j^{t+1}} \sum_{i=1}^{N} q_{ji}^{t+1} \mathbf{x}_i \tag{3}$$

$$\mathbf{\Sigma}_j^{t+1} = \frac{1}{N \beta_j^{t+1}} \sum_{i=1}^{N} q_{ji}^{t+1} (\mathbf{x}_i - \mu_j^{t+1})(\mathbf{x}_i - \mu_j^{t+1})^T$$

$$\lambda_{jk}^{t+1} = \frac{N \beta_j^{t+1}}{\sum_{i=1}^{N} q_{ji}^{t+1} x_{ik}}, \quad \forall k \in \{1, ..., d\}$$

A.2. Computing The Log Sum of Exponents

We encounter a log of a sum of exponents in the optimization of CRAFT. Exponentiation of large numbers can lead to overflow errors and unstable solutions. The following trick is used to avoid numerical errors. Consider a set of real-valued functions $\{g_i\}_{i=1}^m$ with scalar outputs. The log sum of exponents takes the following form:

$$G(\mathbf{x}) = \log \sum_{i=1}^{m} e^{g_i(\mathbf{x})}$$

Let us define a function $g_{max}(\mathbf{x}) = \max_{i \in \{1,2,...,m\}} g_i(\mathbf{x})$. Hence, the above equation can be rewritten as:

$$G(\mathbf{x}) = g_{max}(\mathbf{x}) + \log \sum_{i=1}^{m} \exp\left(g_i(\mathbf{x}) - g_{max}(\mathbf{x})\right)$$

This helps in avoiding the exponentiation of the dominant term and avoids overflow errors. Moreover, if each function outputs a small negative value, the maximum makes each term larger and hence avoids floating point precision errors.

[1] N.B.: For distributions p_θ and $q(z|\mathbf{x})$ in Section A.1 are not related to the distribution of model priors or joint distribution in the CRAFT objective.

B. Additional Results

B.1. Saccade Predictions

Supervised Models. In Figure 1A-B, we highlight the predictions on the source – 'Visual Search' dataset. The EEGNet-LSTM beat the benchmark model in Pyramidal CNNs by 15% on RMSE. Thus, all transfer learning results on saccade amplitude prediction tasks are using the EEGNet-LSTM model.

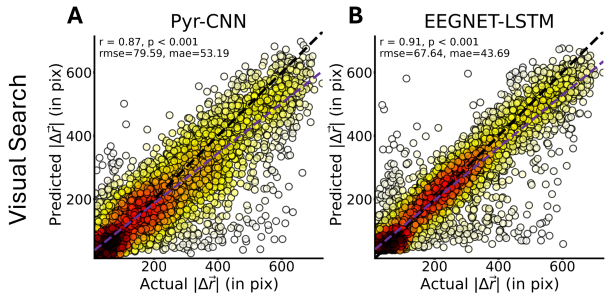


Figure 1. Predictions on the Visual Search Dataset (source dataset) by (A) Pyramidal-CNN (B) EEGNet-LSTM.

Deep Transfer with 100% labeled data. Here, we compare the performance of the proposed method at decoding saccade amplitude with the benchmark and naive transfer learning algorithms. Like in Section **??**, we train a model on the Visual Search dataset. Next, this pretrained model is transferred to the two target datasets. Table 1 summarizes the performance of all the models across 5 seeds. All the models were trained for 100 epochs (batchsize = 128), and only the best-performing model on the held-out cross-validation set was saved and used for evaluation on the test set. Adam optimizer with an initial learning rate of 1e-4 was used. Similarly, we selected the hyperparameter $\alpha \in \{0.01, 0.1, 1.0\}$, associated with the regularization term in equation **??** by a grid search. Empirically, $\alpha = 0.1$ was always favored.

Firstly, we observe that EEGNet-LSTM-based models outperform Pyramidal CNN by a large margin ($> 15\%$), despite having 4x the number of parameters, suggesting support the hypothesis that CNN models designed for images should not be blindly applied to EEG data.[2]

Except for the Pyramidal CNNs applied to the Working Memory dataset (where the model overfits the training data by capturing spurious correlations), our results suggest that CRAFT is model agnostic. The combination of EEGNet-LTSM and CRAFT-based fine-tuning outperforms supervised models by 6% and 15% for the Large Grid and the Working Memory datasets, respectively. Furthermore, CRAFT outperforms naive TL by $\sim 3\%$. Figure 2A shows the predictions for the best model.

Table 1. Effectiveness of CRAFT across two datasets over supervised models and Transfer Learning (TL). The mean performance and the standard error are reported across 5 seeds.

	Large Grid		Working Memory	
Method	**R**	**RMSE** (in pix)	**R**	**RMSE** (in pix)
Naive Baseline	-	149.40 ± 0.00	-	188.82 ± 0.00
Pyr-CNN (Benchmark)	0.91 ± 0.002	63.32 ± 0.82	0.64 ± 0.007	142.65 ± 1.62
Pyr-CNN + TL	0.91 ± 0.002	63.84 ± 0.75	0.63 ± 0.006	141.48 ± 0.60
Pyr-CNN + CRAFT	0.91 ± 0.002	60.95 ± 0.58	0.63 ± 0.005	142.70 ± 0.49
EEGNet-LSTM	0.93 ± 0.002	54.12 ± 0.82	0.72 ± 0.007	133.81 ± 2.38
EEGNet-LSTM + TL	0.93 ± 0.002	51.47 ± 0.63	0.76 ± 0.001	116.36 ± 0.89
EEGNet-LSTM + CRAFT	$\mathbf{0.93 \pm 0.002}$	$\mathbf{50.66 \pm 0.40}$	$\mathbf{0.78 \pm 0.005}$	$\mathbf{113.02 \pm 1.90}$

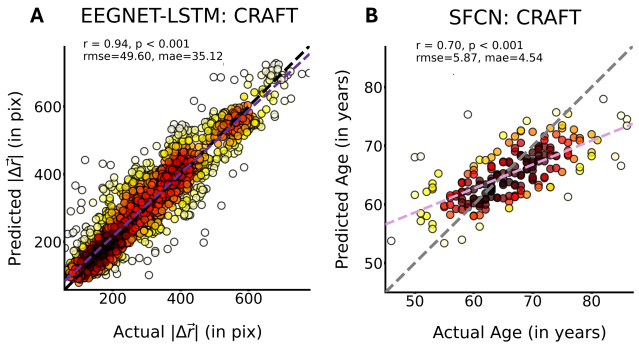


Figure 2. The scatters show the predictions by the best models for the two tasks. (A) Pretrained EEGNet-LSTM model: Transferred to the 'Large Grid' task using CRAFT for saccade amplitude prediction. (B) Pretrained SFCN: Transferred to the TLSA dataset using semi-supervised CRAFT. The CRAFT paradigm allows augmenting the TLSA dataset with unlabeled scans from the RADC dataset.

Next, we show scatter plots for the performance of the Pyramidal CNN-based model on the 'Large Grid Task' in Figure 3A-B. We also compare the best performance of the CRAFT-based transfer of EEGNet-LSTM to its supervised counterpart for the Working Memory dataset (Fig. 3C-D).

Note that, in the working memory dataset, the distribution of the magnitude of intended saccades is bimodal – i.e., participants either had to make a saccade of a constant magnitude or not move their eyes in some catch trials (10%). There is missing support for data for the intermediate magnitudes, so decoding saccades in the dataset is challenging. To estimate the marginal over the labels $p(y)$, we fit the mixture of EMMs and GMMs, as the number of saccades with a magnitude less than 100 pixels is very small.

Semi-supervised Learning. We demonstrate CRAFT as a semi-supervised tool on the EEGEyeNet benchmark's

[2] To avoid variability in the training procedure, we trained the pyramidal CNN the same way we trained the EEGNet-LSTM model, which accounts for the small variation between the numbers reported in Table 1 and the [**?**] The best CRAFT model outperformed the EEGEyeNet benchmark by $\sim 20\%$ on both the datasets.

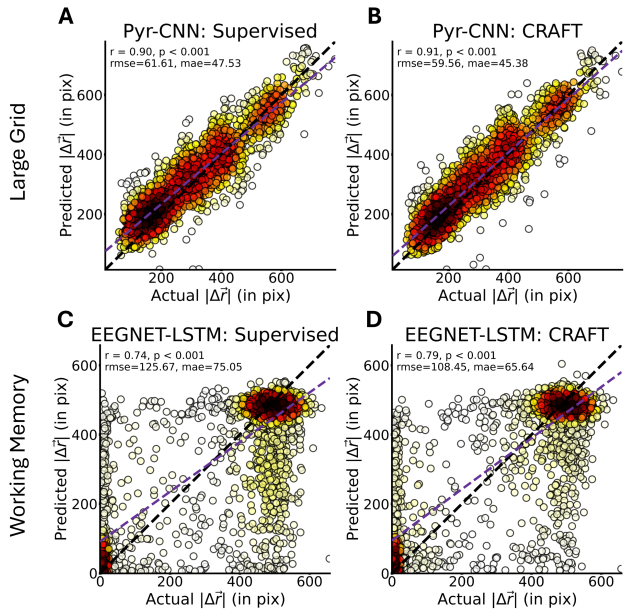


Figure 3. (A-B) Best performing pyramidal CNN-based models on the large grid dataset for each class of training algorithms. (C-D) Best performing (amongst 5 runs) for supervised and CRAFT methods on the Working memory dataset.

Large Grid dataset, i.e., even if a pretrained model is unavailable. The labels from the training data were randomly dropped in a stratified manner such that n_{ul} out of N samples were unlabeled.

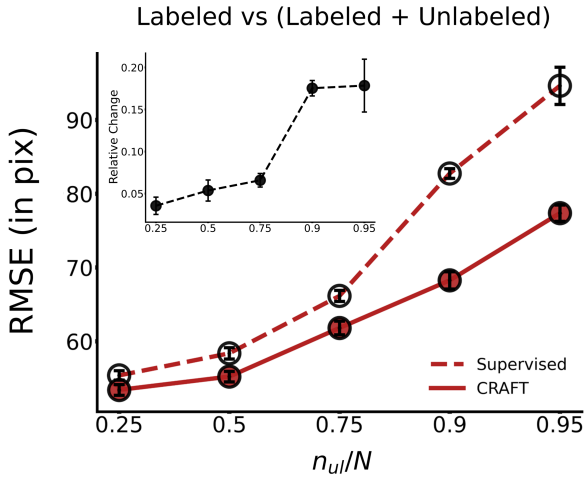


Figure 4. Comparing the performance of CRAFT to supervised training. The error bars are reported across five seeds.

We considered the EEGNet-LSTM model for these experiments since they outperformed Pyramidal CNNs by a large margin (Table 1). Here, like classical semi-supervised learning algorithms, we don't pretrain the model on the Visual Search dataset. By design, the supervised algorithms had to ignore the unlabeled data which CRAFT could use. Models were trained identically to the transfer learning task. The weight associated with the regularizer was fixed to a robust choice of $\alpha = 0.1$.

Figure 4 shows that the CRAFT can be used effectively for semi-supervised learning. The performance in shown in terms of RMSE. The correlation coefficient showed similar trends (Fig. 5A), too. We observe that it outperforms supervised training by $15 - 20\%$, when little labeled data is available ($n_{ul}/N \in \{0.9, 0.95\}$). Although the gap reduces as labeled data increases, CDIST continues to outperform supervised training. Figure 5B-C shows the scatter plots for the predictions by supervised vs CRAFT models when only 5% of the data was labeled.

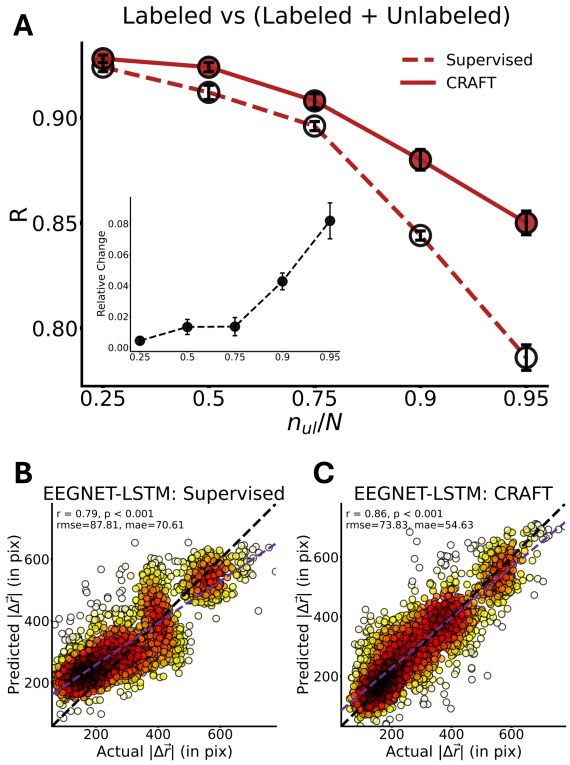


Figure 5. The Large Grid Dataset. (A) Correlation Coefficient achieved by CRAFT to supervised training. The error bars are reported across 5 seeds. (B) Predictions of the supervised model trained on 5% of labelled data (C) Predictions by CRAFT-based EEGNet-LSTM with 5% labeled and remaining unlabeled data.

B.2. Brain Age Predictions

Supervised Results. In Section **??**, we saw that naive inferencing using pretrained SFCN (UKBio) did not perform

as well as CRAFT or fine-tuning. Figure 6A-B shows the scatter plots for the same. We also show the performance of SFCN trained on the TLSA dataset. This reinforces the importance of pretrained models.

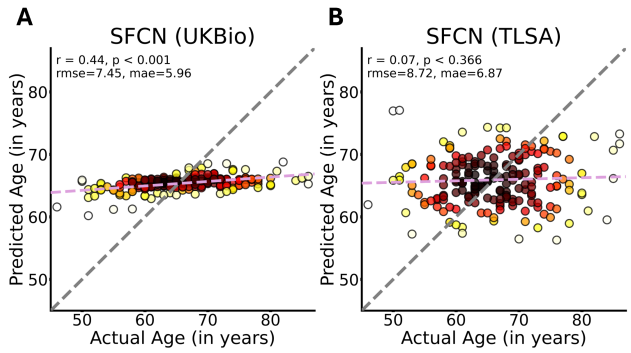


Figure 6. (A) Performance of pretrained SFCN model (on UK Biobank) on the TLSA dataset – naive inferencing. (B) Scatter plot of predictions of the SFCN model when it was trained only on the TLSA dataset – supervised learning.

Deep Transfer with 100% labeled data. The TLSA dataset has only 188 scans, so we avoid hyperparameter searches. All models are trained for 50 epochs (batch size = 4) using the Adam optimizer (initial learning rate = 1e-4), and only the final model is used for inference. We used $\alpha = 0.1$ as the weight associated with the regularisation term in CRAFT, as it was a robust choice for the saccade prediction experiment. Table 2 shows the 4-fold cross-validation performance of all the models (5 seeds). Therefore, each metric is calculated for the prediction on the entire dataset, leaving out one fold at a time. The naive baseline performance on the TLSA dataset is 7.91 years. We observed that training a random SFCN with $\sim$ 3M parameters is infeasible with only 141 training datapoints, and using the pretrained model did not generalize readily due to the target domain (TLSA). Both Transfer Learning and CRAFT were able to mitigate the effect of the distribution shift. The best model was obtained when CRAFT leveraged 500 unlabeled scans from an independent dataset – RADC [**?**], further showing its effectiveness as an SF-SSDA tool. The best model improved RMSE by $\sim 3.3\%$ over TL (Fig. 2B).

Table 2. The performance of the SFCN model on the TLSA dataset, trained using supervised or transfer or CRAFT losses. Unlike other algorithms, CRAFT could use unlabeled (ul) data from the RADC dataset.

Method	**R**	**RMSE** (in years)
Naive Baseline	-	7.91 ± 0.00
SFCN (UKBio)	0.44 ± 0.000	7.46 ± 0.00
SFCN (TLSA)	0.06 ± 0.006	8.88 ± 0.08
SFCN + TL	0.66 ± 0.003	6.14 ± 0.03
SFCN + CRAFT	$\mathbf{0.68 \pm 0.003}$	6.04 ± 0.02
SFCN + CRAFT + RADC	0.68 ± 0.004	$\mathbf{5.94 \pm 0.04}$